COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010/Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	Oct 1 2026	**DATE SUBMITTED:**	Oct 1 2026
NAME:	Regala, Joaquin Luiz B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 10.1 Graphs

1. PROCEDURE OUTPUT

ILO A

Code

```
#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};
// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};
class DiaGraph{
// insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
    newNode->val = value;
    newNode->cost = weight;

    newNode->next = head; // point new node to current head
    return newNode;
    }

    int N; // number of nodes in the graph
public:
    adjNode **head; //adjacency list as array of pointers
    // Constructor
    DiaGraph(graphEdge edges[], int n, int N) {
        // allocate new node
        head = new adjNode*[N]();
        this->N = N;
        // initialize head pointer for all vertices
        for (int i = 0; i < N; ++i)
            head[i] = nullptr;
```

```
            head[i] = nullptr;
        // construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;
            // insert in the beginning
            adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);

            // point head pointer to new node
            head[start_ver] = newNode;
        }
    }
    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++)
            delete[] head[i];
        delete[] head;
    }
};
// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i)
{
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val
            << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}
```

```
// graph implementation
int main()
{
    // graph edges array.
    graphEdge edges[] = {
        // (x, y, w) -> edge from x to y with weight w
        {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
    };
    int N = 6; // Number of vertices in the graph
    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);
    // construct graph
    DiaGraph diagraph(edges, n, N);
    // print adjacency list representation of graph
    std::cout<<"Graph adjacency list "<<std::endl<<"(start_vertex, end_vertex, weight):"<<std::endl;
    for (int i = 0; i < N; i++)
    {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }
    return 0;
}
```

Output

```
[Running] cd "c:\Users\regal\OneDrive\Pictures\VSCODE\
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

Analysis

In this code, I implemented a directed graph using an adjacency list. I used linked nodes to store the vertices connected to each vertex, along with their edge weights. The DiaGraph class creates the graph and adds the edges to the appropriate lists. I then used the display_AdjList() function to show the connections and weights of each vertex.

ILO B.1

Code

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>
template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;
    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
    return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
    return this->weight > e.weight;
    }
};
```

```cpp

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G){
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

template <typename T>
class Graph
{
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N)
    {
    }
    // Return number of vertices in the graph
    auto vertices() const
    {
    return V;
    }
    // Return all edges in the graph
    auto &edges() const
    {
        return edge_list;
    }
    void add_edge(Edge<T> &&e)
    {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
```

```
68      // Returns all outgoing edges from vertex v
69      auto outgoing_edges(size_t v) const
70      {
71          std::vector<Edge<T>> edges_from_v;
72          for (auto &e : edge_list)
73          {
74              if (e.src == v)
75                  edges_from_v.emplace_back(e);
76          }
77          return edges_from_v;
78      }
79      // Overloads the << operator so a graph be written directly to a stream
80      // Can be used as std::cout << obj << std::endl;
81      template <typename U>
82      friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
83  private:
84      size_t V; // Stores number of vertices in graph
85      std::vector<Edge<T>> edge_list;
86  };
87
88  template <typename T>
89  auto depth_first_search(const Graph<T> &G, size_t dest)
90  {
91      std::stack<size_t> stack;
92      std::vector<size_t> visit_order;
93      std::set<size_t> visited;
94      stack.push(1); // Assume that DFS always starts from vertex ID 1
95      while (!stack.empty())
96      {
97          auto current_vertex = stack.top();
98          stack.pop();
99          // If the current vertex hasn't been visited in the past
100         if (visited.find(current_vertex) == visited.end())
101         {
102             visited.insert(current_vertex);
103             visit_order.push_back(current_vertex);
104             for (auto e : G.outgoing_edges(current_vertex))
```

```
                for (auto e : G.outgoing_edges(current_vertex))
                {
                    // If the vertex hasn't been visited, insert it in the stack.
                    if(visited.find(e.dest) == visited.end())
                    {
                        stack.push(e.dest);
                    }
                }
            }
        }
        return visit_order;
    }

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
        return G;
}

template <typename T>
void test_DFS()
{
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run DFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
        std::cout << v << std::endl;
}
int main()
{
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

Output

```
[Running] cd "c:\Users\regal\OneDrive\Pictures\VSCODE\CPE010\basta
1:  {2: 0}, {5: 0},
2:  {1: 0}, {5: 0}, {4: 0},
3:  {4: 0}, {7: 0},
4:  {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:  {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:  {4: 0}, {7: 0}, {8: 0},
7:  {3: 0}, {6: 0},
8:  {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2

[Done] exited with code=0 in 4.575 seconds
```

Analysis

In this code, I used Depth-First Search (DFS) to traverse the graph starting from vertex 1. I used a stack so I could keep exploring one path as far as possible before going back and checking another path. I also used a set to keep track of the vertices that I already visited. The DFS order I got from the code is 1, 5, 4, 8, 6, 7, 3,

Code

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>
template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";

        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
}
```

```cpp

template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const
    {
        return V;
    }
    auto &edges() const
    {
        return edge_list;
    }
    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }
    template <typename T>
    friend std::ostream &operator<<(std::ostream &os, const Graph<T> &G);

```

```
78  private:
79          size_t V;
80          std::vector<Edge<T>> edge_list;
81  };
82
83  template <typename T>
84  auto create_reference_graph()
85  {
86      Graph<T> G(9);
87
88      std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
89      edges[1] = {{2, 2}, {5, 3}};
90      edges[2] = {{1, 2}, {5, 5}, {4, 1}};
91      edges[3] = {{4, 2}, {7, 3}};
92      edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
93      edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
94      edges[6] = {{4, 4}, {7, 4}, {8, 1}};
95      edges[7] = {{3, 3}, {6, 4}};
96      edges[8] = {{4, 5}, {5, 3}, {6, 1}};
97
98      for (auto &i : edges)
99          for (auto &j : i.second)
100             G.add_edge(Edge<T>{i.first, j.first, j.second});
101
102     return G;
103 }
104
105 template <typename T>
106 auto breadth_first_search(const Graph<T> &G, size_t dest)
107 {
108     std::queue<size_t> queue;
109     std::vector<size_t> visit_order;
110     std::set<size_t> visited;
111     queue.push(1); // Assume that BFS always starts from vertex ID 1
112     while (!queue.empty())
113     {
114         auto current_vertex = queue.front();
115         queue.pop();
116         // If the current vertex hasn't been visited in the past
117         if (visited.find(current_vertex) == visited.end())
118         {
```

```
118             {
119                 visited.insert(current_vertex);
120                 visit_order.push_back(current_vertex);
121                 for (auto e : G.outgoing_edges(current_vertex))
122                     queue.push(e.dest);
123             }
124         }
125         return visit_order;
126     }
127
128     template <typename T>
129     void test_BFS()
130     {
131         // Create an instance of and print the graph
132         auto G = create_reference_graph<unsigned>();
133         std::cout << G << std::endl;
134         // Run BFS starting from vertex ID 1 and print the order
135         // in which vertices are visited.
136         std::cout << "BFS Order of vertices: " << std::endl;
137         auto bfs_visit_order = breadth_first_search(G, 1);
138         for (auto v : bfs_visit_order)
139             std::cout << v << std::endl;
140     }
141     int main()
142     {
143         using T = unsigned;
144         test_BFS<T>();
145         return 0;
146     }
```

Output

```
[Running] cd "c:\Users\regal\OneDrive\Pictures\VSCODE\CPE010\basta\HOA10\" && g++ ILOB2.c
1:  {2: 2}, {5: 3},
2:  {1: 2}, {5: 5}, {4: 1},
3:  {4: 2}, {7: 3},
4:  {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:  {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:  {4: 4}, {7: 4}, {8: 1},
7:  {3: 3}, {6: 4},
8:  {4: 5}, {5: 3}, {6: 1},
9:

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

Analysis
In this code, I used Breadth-First Search (BFS) to visit the vertices starting from vertex 1. I used a queue so that the vertices I discover first are the ones I visit first. I also used a set to keep track of the vertices I already visited so I wouldn't visit the same vertex multiple times. The BFS order I got from the code is 1, 2, 5, 4, 8, 3, 6, 7.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.
Answer: The most helpful algorithm is Depth-First Search (DFS). It visits a vertex and continues exploring deeper vertices until there are no more unvisited vertices, then backtracks and explores other vertices.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.
Answer: The equivalent of DFS in tree traversal is Preorder Traversal. Both explore a node first before recursively/iteratively exploring its child or neighboring nodes. DFS commonly uses a stack, while tree preorder follows the order Root → Left → Right.

3. In the performed code, what data structure is used to implement the Breadth First Search?
Answer: The data structure used to implement Breadth-First Search (BFS) is a Queue.

4. How many times can a node be visited in the BFS?
Answer: A node can be visited only once in the BFS because the code uses a visited set to prevent the same vertex from being processed again.

CONCLUSION:
In this activity, I learned how different graph implementations and traversal methods work. By working with the adjacency list, BFS, and DFS codes, I was able to see how each method visits and organizes vertices differently. I also learned that BFS uses a queue and visits vertices level by level, while DFS uses a stack and explores deeper before backtracking. Overall, this activity helped me understand how graph traversal works in actual code and how these methods can be applied to different situations.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **None**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **None**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **None**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.